

CHAPTER 4

Terraform State Management

How Terraform tracks infrastructure, why remote state matters, and how to secure and lock it with S3 + DynamoDB.



4.1 WHAT IS TERRAFORM STATE?

Terraform's memory of your infrastructure



Tracks resource dependencies

Knows what must be created, updated, or destroyed in what order.



Maps config to real infrastructure

Connects each resource block to the actual object in the cloud.



Improves performance

Caches attributes so Terraform doesn't re-query every resource on every run.



Enables collaboration

A shared remote state lets a whole team work against the same source of truth.

4.1 WHAT IS TERRAFORM STATE?

Inside terraform.tfstate

```
terraform.tfstate
{
  "resources": [
    {
      "type": "aws_s3_bucket",
      "name": "my_bucket",
      "instances": [
        {
          "attributes": {
            "bucket": "my-terraform-bucket-123",
            "arn": "arn:aws:s3:::my-terraform-bucket-123"
          }
        }
      ]
    }
  ]
}
```



What this maps to

type / name

Which resource block in your .tf config this entry belongs to.

attributes

The real-world values Terraform read back from AWS after creation.

arn

The unique AWS identifier — proof this state entry is bound to one specific bucket.

Where should your state file live?



Local State (default)

`terraform apply` → `terraform.tfstate`



Easy to use — zero setup required



Hard to share across a team



Risk of accidental deletion



Remote State (teams)

S3 • Azure Blob • GCS • Terraform Cloud



Shared access for the whole team



Prevents conflicting updates



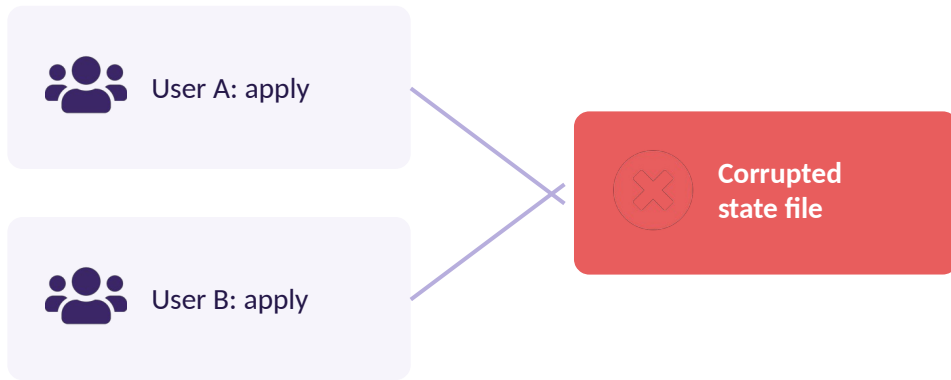
Secure, durable storage



Requires extra setup

Why concurrent applies are dangerous

Without locking



State Locking

Locking prevents concurrent modifications by locking the state file while an apply is in progress — the second user must wait.



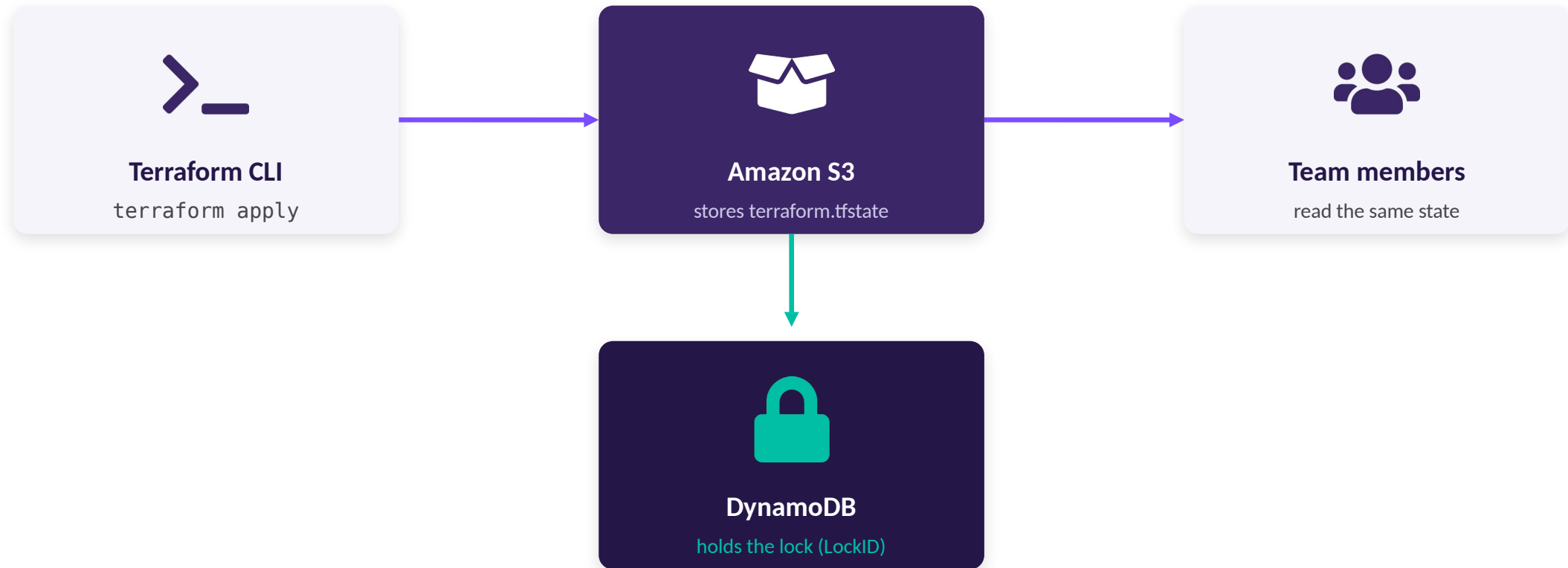
DynamoDB for AWS locking

When state is stored in S3, a DynamoDB table holds the active lock record — Terraform checks it before every plan or apply.



Encryption + versioning add defense in depth

The target architecture



backend.tf tells Terraform where the S3 bucket and DynamoDB table live — every teammate's CLI points at the same backend.

Step 1–2 · Create the bucket and lock table

1 Create the S3 bucket

```
$ aws s3api create-bucket \  
  --bucket my-terraform-state-bucket \  
  --region us-east-1
```



Enable versioning to protect the state file:

```
$ aws s3api put-bucket-versioning \  
  --bucket my-terraform-state-bucket \  
  --versioning-configuration Status=Enabled
```

2 Create the DynamoDB lock table

```
$ aws dynamodb create-table \  
  --table-name terraform-lock \  
  --attribute-definitions \  
    AttributeName=LockID,AttributeType=S \  
  --key-schema \  
    AttributeName=LockID,KeyType=HASH \  
  --billing-mode PAY_PER_REQUEST
```



LockID is the unique key each state lock is stored under.

Step 3–4 · Point Terraform at the backend

3 Configure backend.tf

```
backend.tf

terraform {
  backend "s3" {
    bucket      = "my-terraform-state-bucket"
    key         = "terraform.tfstate"
    region      = "us-east-1"
    dynamodb_table = "terraform-lock"
    encrypt     = true
  }
}
```

Every field here is required for locking to work — omit `dynamodb_table` and Terraform will store state in S3 with no lock protection.

4 Initialize & apply

```
$ terraform init
```

Moves local state into S3.

```
$ terraform apply -auto-approve
```

Locks the state file in DynamoDB while applying.



From this point on, everyone on the team runs against the same remote, locked state.

Confirm it actually worked



Check the state file in S3

```
$ aws s3 ls s3://my-terraform-state-bucket
```



Expected result: terraform.tfstate appears in the bucket listing.

This confirms Terraform successfully migrated state off your laptop and into shared, durable storage.



Check state locks in DynamoDB

```
$ aws dynamodb scan --table-name terraform-lock
```



Expected result: shows an active lock item while an apply is running, empty otherwise.

If a second apply is attempted while a lock is held, Terraform will refuse to proceed and report who holds the lock.

From local files to a locked, shared source of truth



Terraform state & why it's needed

Maps config to real resources, tracks dependencies, and caches attributes.



Local vs. remote state

Local is simple but unsafe for teams; remote backends like S3 enable collaboration.



State locking with DynamoDB

Prevents two people from applying at once and corrupting shared state.



Stored Terraform state in S3

Versioned, encrypted, and locked — a durable source of truth for the whole team.

9-slide chapter complete — up next: Chapter 5